

✓ Experiment-2: Basic Text Preprocessing

This notebook contains simple and clear Python programs for **Experiment 2**:

- **Experiment 2.1:** Basic Text Preprocessing (lowercase conversion, punctuation removal, number removal, extra whitespace cleaning, and count statistics).
- **Experiment 2.2:** Word & Sentence Tokenization (3 separate implementations: without NLTK/spaCy, using NLTK, and using spaCy).
- **Experiment 2.3:** Tokenization & Stop Words Removal (displaying original tokens, removed stop words separately, and filtered tokens).

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

✓ Experiment 2.1: Basic Text Preprocessing

Write a Python program to perform basic text preprocessing on a given data. The program should:

- Convert text to lowercase. *(Also display how many uppercase letters)*
- Remove punctuation. *(Also display how many different types of punctuations)*
- Remove numbers. *(Also display how many numbers)*
- Remove extra whitespace. *(Display how much extra whitespace is present)*
- Display the cleaned text.

```
import os
import string
import re

# Read data from 2.1_text_data.txt
file_path_21 = '/content/drive/MyDrive/NLP_Lab/Experiment-2/2.1_text_data.txt'
if os.path.exists(file_path_21):
    with open(file_path_21, 'r', encoding='utf-8') as f:
        text_21 = f.read()
else:
    text_21 = """Natural Language Processing (NLP) is a branch of Artificial Intelligence. It helps computers understand an
In 2026, NLP is used in many applications such as chatbots, machine translation, sentiment analysis, and text summarization!
Text preprocessing is an important step in NLP. It includes several operations such as cleaning text, removing punctuation
For example, a chatbot may process 1000+ customer messages every day. These messages can contain CAPITAL letters, numbers li
The main goal of text preprocessing is to convert raw text into a clean and consistent format. This makes the text easier to

print("=== ORIGINAL TEXT ===")
print(text_21)
print("-" * 50)

# 1. Count Uppercase Letters
uppercase_count = sum(1 for char in text_21 if char.isupper())

# 2. Count Punctuation types & remove punctuation
punctuated_chars = [char for char in text_21 if char in string.punctuation]
unique_punctuations = set(punctuated_chars)
punctuation_types_count = len(unique_punctuations)
no_punct_text = text_21.translate(str.maketrans('', '', string.punctuation))

# 3. Count Numbers & remove numbers
numbers_found = re.findall(r'\d+', text_21)
numbers_count = len(numbers_found)
no_numbers_text = re.sub(r'\d+', '', no_punct_text)

# 4. Count Extra Whitespace & clean text
extra_spaces_count = len(re.findall(r'\s{2,}', no_numbers_text))
cleaned_text = ' '.join(no_numbers_text.lower().split())

# Display Statistics
print("\n=== STATISTICS ===")
print(f"Uppercase letters count      : {uppercase_count}")
print(f"Punctuation types count      : {punctuation_types_count} -> {sorted(list(unique_punctuations))}")
print(f"Numbers found count           : {numbers_count} -> {numbers_found}")
print(f"Extra whitespace instances    : {extra_spaces_count}")

print("\n=== CLEANED TEXT ===")
print(cleaned_text)
```

```

=== ORIGINAL TEXT ===
Natural Language Processing (NLP) is a branch of Artificial Intelligence. It helps computers understand and process human language.

In 2026, NLP is used in many applications such as chatbots, machine translation, sentiment analysis, and text summarization!

Text preprocessing is an important step in NLP. It includes several operations such as cleaning text, removing punctuation, and normalizing text.

For example, a chatbot may process 1000+ customer messages every day. These messages can contain CAPITAL letters, numbers like 2026, and special characters.

The main goal of text preprocessing is to convert raw text into a clean and consistent format. This makes the text easier to process and analyze.
-----

=== STATISTICS ===
Uppercase letters count      : 32
Punctuation types count     : 6 -> ['!', '(', ')', '+', ',', '.', ':']
Numbers found count         : 3 -> ['2026', '1000', '12345']
Extra whitespace instances   : 10

=== CLEANED TEXT ===
natural language processing nlp is a branch of artificial intelligence it helps computers understand and process human language

```

✓ Experiment 2.2: Word and Sentence Tokenization

Write a Python program to perform word tokenization and sentence tokenization on a given text. Display the resulting words and sentences separately. *(Three separate codes: without using NLTK/spaCy, using NLTK, and using spaCy)*

✓ 1. Tokenization Without Using NLTK or spaCy (Pure Python & Regex)

```

import os
import re

# Read data from 2.2_tokenization_data.txt
file_path_22 = '/content/drive/MyDrive/NLP_Lab/Experiment-2/2.2_tokenization_data.txt'
if os.path.exists(file_path_22):
    with open(file_path_22, 'r', encoding='utf-8') as f:
        text_22 = f.read()
else:
    text_22 = """Natural Language Processing (NLP) is a fascinating field of Artificial Intelligence. It enables computers
    Text tokenization is one of the first steps in NLP. Sentence tokenization divides a paragraph into individual sentences. We
    For example, a chatbot receives thousands of messages every day. The system processes these messages and identifies useful

# Sentence Tokenization using Regex
sentences_custom = [s.strip() for s in re.split(r'(?<=[!?.])\s+', text_22.strip()) if s.strip()]

# Word Tokenization using Regex
words_custom = re.findall(r'\w+|[\^\w\s]', text_22)

print("=== SENTENCES (Without NLTK/spaCy) ===")
print(f"Total Sentences: {len(sentences_custom)}")
for i, sent in enumerate(sentences_custom, 1):
    print(f"{i}. {sent}")

print("\n=== WORDS / TOKENS (Without NLTK/spaCy) ===")
print(f"Total Words: {len(words_custom)}")
print(words_custom)

=== SENTENCES (Without NLTK/spaCy) ===
Total Sentences: 9
1. Natural Language Processing (NLP) is a fascinating field of Artificial Intelligence.
2. It enables computers to understand human language.
3. NLP is used in many real-world applications, such as chatbots, machine translation, sentiment analysis, and text summarization.
4. Text tokenization is one of the first steps in NLP.
5. Sentence tokenization divides a paragraph into individual sentences.
6. Word tokenization divides each sentence into individual words and tokens.
7. For example, a chatbot receives thousands of messages every day.
8. The system processes these messages and identifies useful information.
9. This information can then be used to provide better responses to users.

=== WORDS / TOKENS (Without NLTK/spaCy) ===
Total Words: 112
['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'fascinating', 'field', 'of', 'Artificial', 'Intelligence'

```

✓ 2. Tokenization Using NLTK

```

import nltk
from nltk.tokenize import sent_tokenize, word_tokenize

# Download NLTK tokenizer resources

```

```

nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)

# Sentence Tokenization using NLTK
sentences_nltk = sent_tokenize(text_22)

# Word Tokenization using NLTK
words_nltk = word_tokenize(text_22)

print("=== SENTENCES (Using NLTK) ===")
print(f"Total Sentences: {len(sentences_nltk)}")
for i, sent in enumerate(sentences_nltk, 1):
    print(f"{i}. {sent}")

print("\n=== WORDS / TOKENS (Using NLTK) ===")
print(f"Total Words: {len(words_nltk)}")
print(words_nltk)

=== SENTENCES (Using NLTK) ===
Total Sentences: 9
1. Natural Language Processing (NLP) is a fascinating field of Artificial Intelligence.
2. It enables computers to understand human language.
3. NLP is used in many real-world applications, such as chatbots, machine translation, sentiment analysis, and text summariz
4. Text tokenization is one of the first steps in NLP.
5. Sentence tokenization divides a paragraph into individual sentences.
6. Word tokenization divides each sentence into individual words and tokens.
7. For example, a chatbot receives thousands of messages every day.
8. The system processes these messages and identifies useful information.
9. This information can then be used to provide better responses to users.

=== WORDS / TOKENS (Using NLTK) ===
Total Words: 110
['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'fascinating', 'field', 'of', 'Artificial', 'Intelligence'

```

3. Tokenization Using spaCy

```

try:
    import spacy
    try:
        nlp = spacy.load('en_core_web_sm')
    except OSError:
        import spacy.cli
        spacy.cli.download('en_core_web_sm')
        nlp = spacy.load('en_core_web_sm')

    doc_spacy = nlp(text_22)
    sentences_spacy = [sent.text.strip() for sent in doc_spacy.sents]
    words_spacy = [token.text for token in doc_spacy]

    print("=== SENTENCES (Using spaCy) ===")
    print(f"Total Sentences: {len(sentences_spacy)}")
    for i, sent in enumerate(sentences_spacy, 1):
        print(f"{i}. {sent}")

    print("\n=== WORDS / TOKENS (Using spaCy) ===")
    print(f"Total Words: {len(words_spacy)}")
    print(words_spacy)
except ImportError:
    print("spaCy library is not installed.")
    print("Run: !pip install spacy && python -m spacy download en_core_web_sm")

=== SENTENCES (Using spaCy) ===
Total Sentences: 9
1. Natural Language Processing (NLP) is a fascinating field of Artificial Intelligence.
2. It enables computers to understand human language.
3. NLP is used in many real-world applications, such as chatbots, machine translation, sentiment analysis, and text summariz
4. Text tokenization is one of the first steps in NLP.
5. Sentence tokenization divides a paragraph into individual sentences.
6. Word tokenization divides each sentence into individual words and tokens.
7. For example, a chatbot receives thousands of messages every day.
8. The system processes these messages and identifies useful information.
9. This information can then be used to provide better responses to users.

=== WORDS / TOKENS (Using spaCy) ===
Total Words: 114
['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'fascinating', 'field', 'of', 'Artificial', 'Intelligence'

```

Experiment 2.3: Tokenization and Stop Words Removal

Write a Python program to tokenize a text and remove commonly occurring stop words such as `is`, `the`, `a`, `an`, `in`, `of`, `and`, `to`, etc. Display the original and filtered tokens. (Also, display Stop Words Separately).

```
import os
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download NLTK datasets
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)

# Read data from 2.3_clean_data.txt
file_path_23 = '/content/drive/MyDrive/NLP_Lab/Experiment-2/2.3_clean_data.txt'
if os.path.exists(file_path_23):
    with open(file_path_23, 'r', encoding='utf-8') as f:
        text_23 = f.read()
else:
    text_23 = """Natural Language Processing is an important field of Artificial Intelligence.
It is used in many applications such as chatbots, machine translation, sentiment analysis, and text summarization.
The main purpose of NLP is to enable computers to understand and process human language.
A computer can analyze a large amount of text in a short period of time.
The removal of stop words is one of the common steps in text preprocessing.
Stop words are commonly occurring words such as the, is, a, an, in, of, and, to, for, on, with, from, by, this, that, and i
Removing these words can reduce the size of the text and improve the efficiency of some NLP applications."""

# 1. Original Tokens
original_tokens = word_tokenize(text_23)

# 2. Stop words set (NLTK english + target stop words: is, the, a, an, in, of, and, to, etc.)
nltk_stop_words = set(stopwords.words('english'))
target_stop_words = {'is', 'the', 'a', 'an', 'in', 'of', 'and', 'to', 'for', 'on', 'with', 'from', 'by', 'this', 'that', 'i'}
stop_words_set = nltk_stop_words.union(target_stop_words)

# 3. Separate stop words and filtered tokens
removed_stop_words = []
filtered_tokens = []

for token in original_tokens:
    if token.lower() in stop_words_set and token.isalnum():
        removed_stop_words.append(token)
    else:
        filtered_tokens.append(token)

unique_stop_words_removed = sorted(list(set([w.lower() for w in removed_stop_words])))

# 4. Display Results
print("=== ORIGINAL TOKENS ===")
print(f"Total Original Tokens: {len(original_tokens)}")
print(original_tokens)

print("\n=== STOP WORDS REMOVED (SEPARATELY) ===")
print(f"Total Stop Words Removed: {len(removed_stop_words)} (Unique: {len(unique_stop_words_removed)})")
print(unique_stop_words_removed)

print("\n=== FILTERED TOKENS (AFTER REMOVAL) ===")
print(f"Total Filtered Tokens: {len(filtered_tokens)}")
print(filtered_tokens)
```

```
=== ORIGINAL TOKENS ===
Total Original Tokens: 138
['Natural', 'Language', 'Processing', 'is', 'an', 'important', 'field', 'of', 'Artificial', 'Intelligence', '.', 'It', 'is',

=== STOP WORDS REMOVED (SEPARATELY) ===
Total Stop Words Removed: 57 (Unique: 22)
['a', 'an', 'and', 'are', 'as', 'by', 'can', 'for', 'from', 'in', 'is', 'it', 'of', 'on', 'some', 'such', 'that', 'the', 'th

=== FILTERED TOKENS (AFTER REMOVAL) ===
Total Filtered Tokens: 81
['Natural', 'Language', 'Processing', 'important', 'field', 'Artificial', 'Intelligence', '.', 'used', 'many', 'applications']
```

